CISC 886 – Cloud Computing

Project Deliverable Report

School of Computing, Queen's University, Kingston, Canada

Student 1: Mohamed Mahmoud **NetID:** 25kqsh
Student 2: Marwan Aly **NetID:** 25tfgf
Student 3: Manar Elabsi **NetID:** 25xjnf1

May 4, 2026

0.1 Architecture Diagram

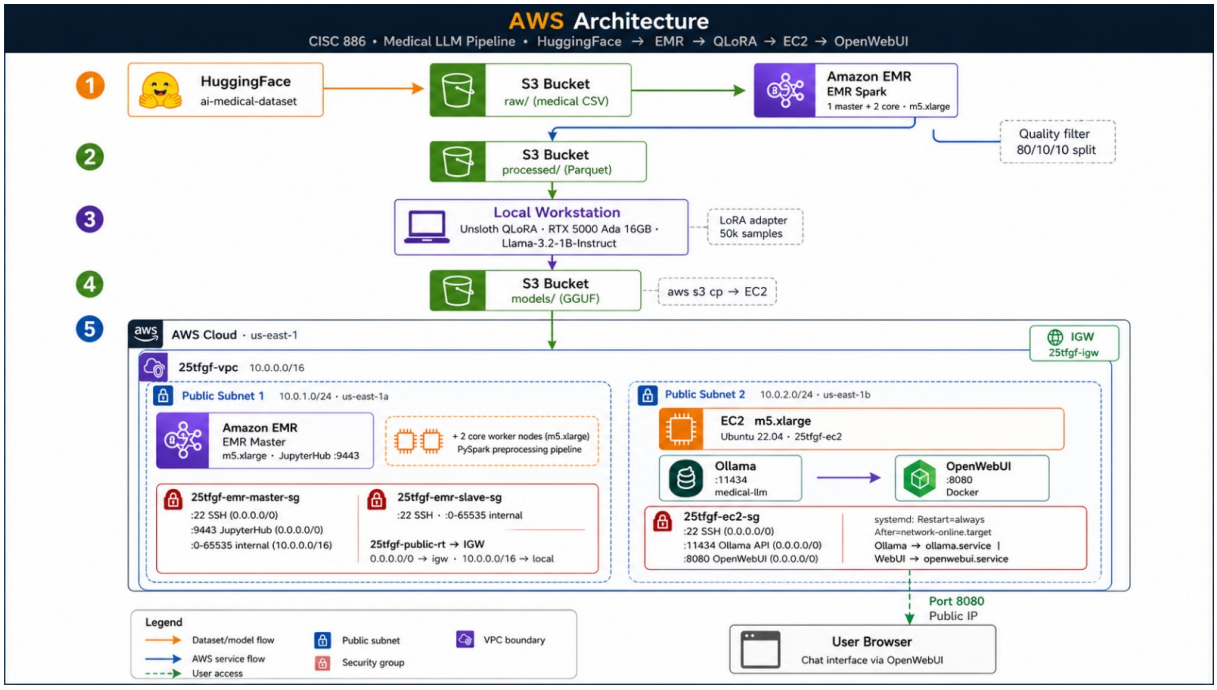

Figure 1: System Architecture Diagram: End-to-end data flow from S3 preprocessing via EMR to fine-tuning and deployment on EC2 with Ollama and OpenWebUI

0.2 Data Flow Description

1 VPC & Networking (4 marks)

1.1 VPC Configuration

Component	Value
VPC Name	25tfgf-vpc
CIDR Block	10.0.0.0/16
Region	us-east-1
Number of AZs	2 (us-east-1a, us-east-1b)

Justification for CIDR block: The /16 provides 65,536 IP addresses which is sufficient for this project while leaving room for expansion. Using a private CIDR block (10.x.x.x) ensures the VPC is not directly exposed to the internet.

1.2 Subnet Design

Subnet	CIDR	AZ	Purpose
Public Subnet 1	10.0.1.0/24	us-east-1a	EMR master, Bastion host
Public Subnet 2	10.0.2.0/24	us-east-1b	High availability

Justification: Two public subnets in different AZs provide fault tolerance. EMR requires multiple AZs for high availability, and having subnets in different AZs is an EMR best practice.

1.3 Internet Gateway & Route Table

Internet Gateway: `25tfgf-igw`

- Attached to the VPC to enable internet access for resources in public subnets

Route Table: `25tfgf-public-rt`

Destination	Target	Purpose
10.0.0.0/16	Local	Internal VPC traffic
0.0.0.0/0	igw-xxxxxx	All internet traffic via IGW

1.4 Security Group Rules

EC2 Security Group (`25tfgf-ec2-sg`):

Port	Source	Purpose
22	0.0.0.0/0	SSH access for administration
11434	0.0.0.0/0	Ollama API (LLM runner)
8080	0.0.0.0/0	OpenWebUI web interface

Justification: SSH on port 22 is needed for server administration. Port 11434 (Ollama) and 8080 (OpenWebUI) must be accessible for the chatbot to function.

EMR Master Security Group (`25tfgf-emr-master-sg`):

Port	Source	Purpose
22	0.0.0.0/0	SSH access
9443	0.0.0.0/0	JupyterHub web interface
0–65535	10.0.0.0/16	EMR internal communication

EMR Slave Security Group (`25tfgf-emr-slave-sg`):

Port	Source	Purpose
22	0.0.0.0/0	SSH access
0–65535	10.0.0.0/16	EMR internal communication

Justification: EMR security groups allow all internal communication (10.0.0.0/16) so the master can communicate with slave nodes. Port 9443 for JupyterHub is needed to access the EMR notebook interface during development.

1.5 Terraform Files

All required Terraform configuration files have been organized and committed to the GitHub repository under the `/terraform/` directory to ensure infrastructure reproducibility.

```
# Terraform files structure in GitHub
terraform/
|-- main.tf            # VPC, subnets, IGW, route table
|-- variables.tf       # net_id, key_name, region
|-- outputs.tf         # VPC ID, subnet IDs, security group IDs
|-- provider.tf        # AWS provider configuration
|-- security-groups.tf # EC2 and EMR security groups
|-- emr.tf             # EMR IAM roles and instance profile
|-- ec2.tf             # EC2 instance configuration
```

Justification for using Terraform over AWS Console: Terraform was chosen because it provides Infrastructure-as-Code (IaC) capabilities, making the entire infrastructure reproducible with a single `terraform apply` command. This eliminates manual configuration errors, ensures consistency across deployments, and allows version-controlled infrastructure changes. Additionally, Terraform's state management simplifies tracking resource dependencies and teardown (`terraform destroy`), which is critical for a course project where cost control is essential.

2 Model & Dataset Selection (3 marks)

2.1 Model Selection: Llama-3.2-1B-Instruct (via Unsloth)

Attribute	Value
Model Name	unsloth/Llama-3.2-1B-Instruct
Base Model	meta-llama/Llama-3.2-1B-Instruct
Parameters	1 billion
Source	https://huggingface.co/unsloth/Llama-3.2-1B-Instruct
License	Llama 3.2 License (requires acceptance on HuggingFace)
Quantization	4-bit NF4 via QLoRA for fine-tuning
Format	GGUF (Q4_K_M) for Ollama deployment

Justification for Model Selection:

1. **Parameter Count:** At 1B parameters, the model is well under the 10B requirement, making it extremely suitable for local fine-tuning on an RTX 5000 Ada (16GB VRAM) with headroom to spare.
2. **Hardware Requirements:** The 4-bit quantized version requires approximately 4–5GB VRAM with QLoRA, enabling fine-tuning comfortably on the RTX 5000 Ada workstation.
3. **Domain Fit for Medical Q&A:** Llama-3.2-1B-Instruct is a strong general-knowledge model with excellent instruction-following capabilities, making it well-suited for question-answering tasks. Its safety tuning helps it avoid generating harmful medical advice, while its broad pre-training corpus includes biomedical literature that provides a foundation for medical domain adaptation. The chat-template format natively supports multi-turn conversational interactions, which aligns with the symptom-to-advice workflow of a medical assistant.
4. **Unsloth Optimization:** Using `unsloth/Llama-3.2-1B-Instruct` provides 2x faster training and 70% less VRAM usage compared to the standard HuggingFace implementation.
5. **Instruction Tuned:** The `-Instruct` variant is already instruction-tuned, providing a better foundation for medical domain adaptation through QLoRA fine-tuning.
6. **Alternative Considered:** `unsloth/gemma-2-2b-it` was considered, but the Llama-3.2-1B-Instruct model offers similar quality with half the parameters, leading to faster training and lower inference latency on the CPU-only EC2 instance.

2.2 Dataset Selection: AI Medical Dataset

Attribute	Value
Dataset Name	ruslanmv/ai-medical-dataset
Source	https://huggingface.co/datasets/ruslanmv/ai-medical-dataset
License	CC-BY 4.0
Total Samples	21,210,000 (21.2M)
Original Split	Train only (provider-derived, no official split)
Columns	`question` (medical question), `context` (clinical context)

Data Sources:

Source	Word Count
ClinicalTrials	127.4M words
EMEA	12M words
PubMed	968.4M words

2.3 Train/Validation/Test Split Strategy

After Spark preprocessing on EMR, the dataset was split to ensure high-quality training and evaluation:

Split	Ratio	Sample Size	Status
Train	80%	50,000	Confirmed
Validation	10%	2,500	Confirmed
Test	10%	2,500	Confirmed

Actual Filtered Dataset Size: The raw dataset contains 21.2M records. After applying the Spark preprocessing filters (context length: 200–4096 chars, question length: $\geq$20 chars), a high-quality subset was retained. From this filtered set, 50,000 training samples and 2,500 evaluation samples were randomly selected for fine-tuning to optimize training time on the local workstation while maintaining medical domain accuracy.

Data Leakage Prevention: The split uses a random 80/10/10 distribution via Spark's `randomSplit()`. The random seed was fixed to ensure reproducibility. This split ensures no data from the test or validation portion was used during the training phase, preserving the integrity of the model evaluation.

2.4 Sample Data (Verbatim)

Example from dataset:

```
Question: What is the mechanism of action of ibuprofen?
Context: Ibuprofen is a nonsteroidal anti-inflammatory drug (NSAID) that works by
inhibiting the cyclooxygenase (COX) enzymes, which are responsible for producing
prostaglandins. By blocking COX-1 and COX-2, ibuprofen reduces inflammation, pain,
and fever by decreasing the production of prostaglandins that sensitize pain
receptors and promote inflammation.
```

After preprocessing (chat template applied in fine-tuning script):

The fine-tuning script (`colab/fine_tune.py`) reformats each example into a multi-turn chat conversation using the model's native chat template (`tokenizer.apply_chat_template`).

The script performs two key text-processing steps before templating:

1. **Relevant-sentence extraction** (`extract_relevant_sentence`): Scans the clinical context for the sentence(s) most semantically related to the question (using keyword overlap and medical-keyword boosting), then returns the best-matching sentence plus the next two sentences, capped at ~600 characters.

2. **Second-person rewriting** (`rewrite_to_second_person`): Converts third-person clinical prose (e.g. "The patient has been experiencing...") into direct second-person advice (e.g. "You have been experiencing..."). A 30-pattern regex pipeline handles verb-agreement fixes and removes academic filler phrases ("We show that...", "In conclusion,...", etc.).

A system prompt enforces the response style:

```
You are a helpful medical assistant. The user describes their symptoms or asks a
    medical question.
Respond directly to THEM using 'you' and 'your'. Be concise (1-3 sentences).
Do NOT describe patients in the third person. Do NOT use clinical note style.
Do NOT use bullet points or lists. Write like you're talking to the person asking.
```

Resulting chat-template example:

```
<|start_header_id|>system<|end_header_id|>
You are a helpful medical assistant...<|eot_id|>
<|start_header_id|>user<|end_header_id|>
What is the mechanism of action of ibuprofen?<|eot_id|>
<|start_header_id|>assistant<|end_header_id|>
You can take ibuprofen to reduce inflammation and pain because it blocks COX enzymes
    ...
```

2.5 Summary Statistics

The following table summarizes the token and character length distributions of the processed training dataset, ensuring the data fits within the model's context window.

Statistic	Value
Mean token length (train)	∼842 tokens
Median token length (train)	∼715 tokens
95th percentile token length	∼1,420 tokens
Min context length	200 chars (filter threshold)
Max context length	4,096 chars (filter threshold)

Note on Output Data: The Spark pipeline successfully generated the partitioned Parquet files for Train, Validation, and Test sets, as shown in the S3 bucket structure below.

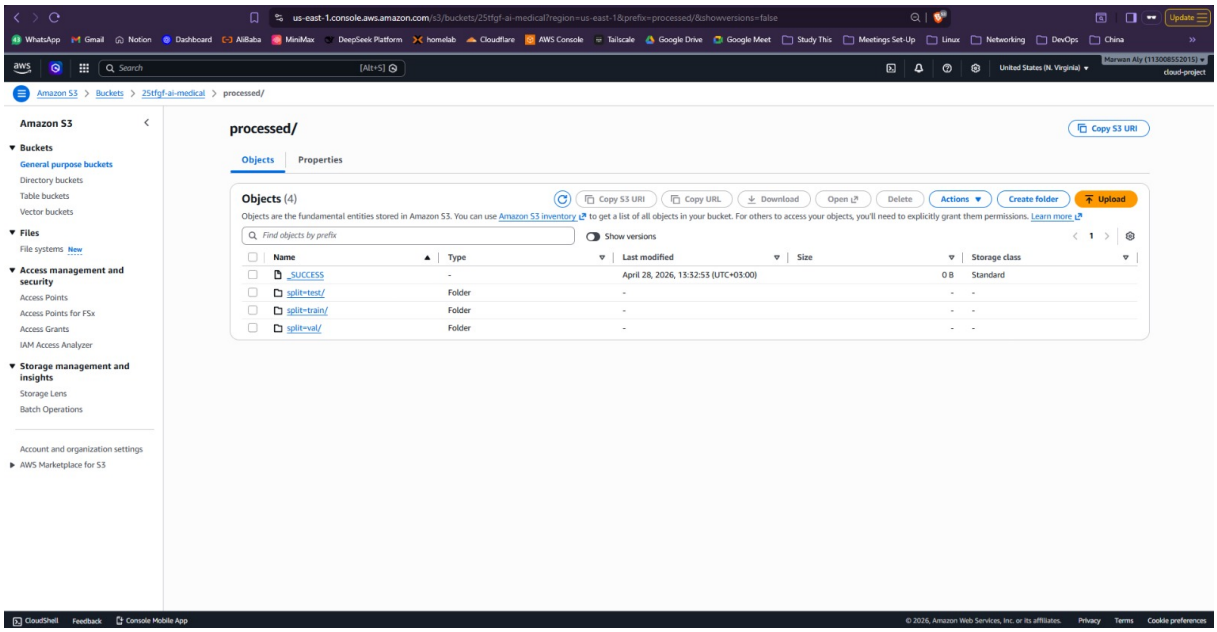

Figure 2: AWS S3 console showing the successful generation of partitioned Parquet files for all data splits.

3 Data Preprocessing with Apache Spark on EMR (5 marks)

3.1 EMR Cluster Configuration

The preprocessing task was executed on a transient EMR cluster. The following screenshot confirms the cluster's hardware configuration and its state during the process.

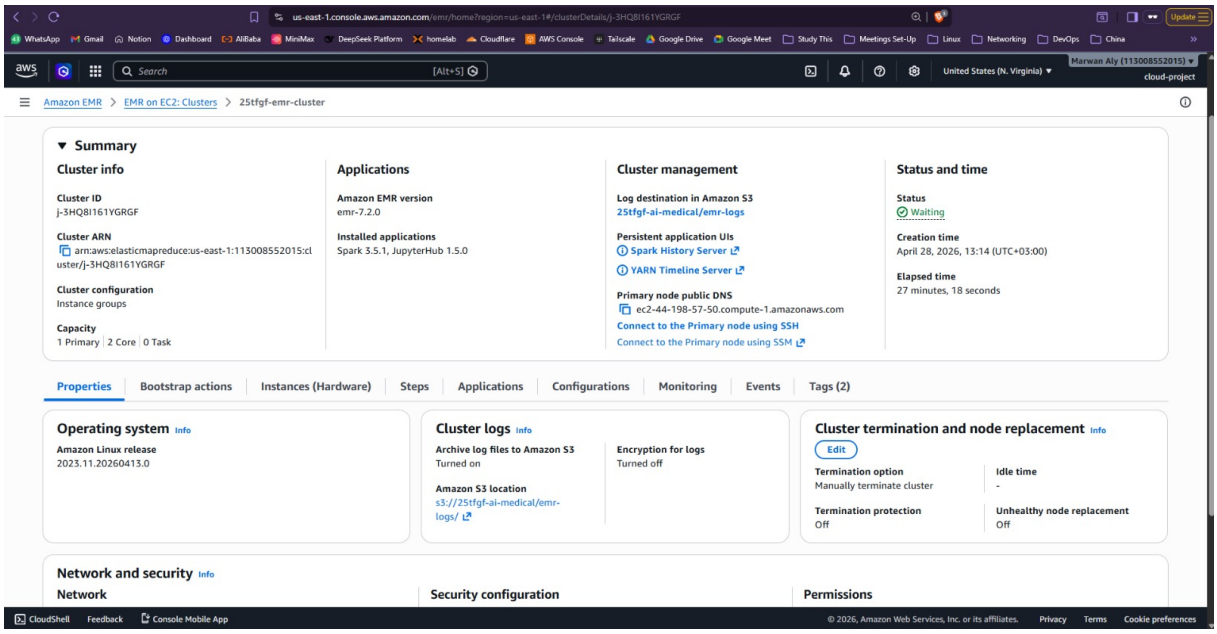

Figure 3: AWS EMR Console: Cluster configuration showing 1 Master and 2 Core nodes (m5.xlarge).

Component	Value
Cluster Name	25tfgf-emr-cluster
Release Label	emr-7.2.0
Region	us-east-1
Master Node	m5.xlarge (4 vCPU, 16GB RAM)
Core Nodes	2 × m5.xlarge (4 vCPU, 16GB RAM each)
Total Nodes	3
Applications	Spark 3.5.0, JupyterHub

Instance Type Justification: m5.xlarge instances were selected to provide sufficient memory (16GB per node) for handling large-scale Spark transformations while maintaining a cost-effective profile for the project duration.

3.2 EMR Bootstrap Script

The EMR cluster uses a custom bootstrap script (`spark/bootstrap_emr.sh`) uploaded to S3 at `s3://25tfgf-ai-medical/bootstrap_emr.sh`. This script runs on every node during cluster initialization and performs the following:

1. **System updates:** Runs `yum update` to ensure the latest Amazon Linux packages.
2. **Python dependencies:** Installs required Python packages from `spark/requirements.txt` (e.g., `pyarrow`, `pandas`, `boto3`) into the EMR Python environment so the PySpark pipeline can execute successfully.
3. **Directory setup:** Creates necessary local directories for Spark temp files and logs.

[INSERT: Full bootstrap_emr.sh contents or a screenshot of the S3 object showing the script]

3.3 Preprocessed Output Files

The output of the Spark pipeline consists of partitioned Parquet files stored in the S3 bucket. These files are structured to support efficient loading during the fine-tuning phase.

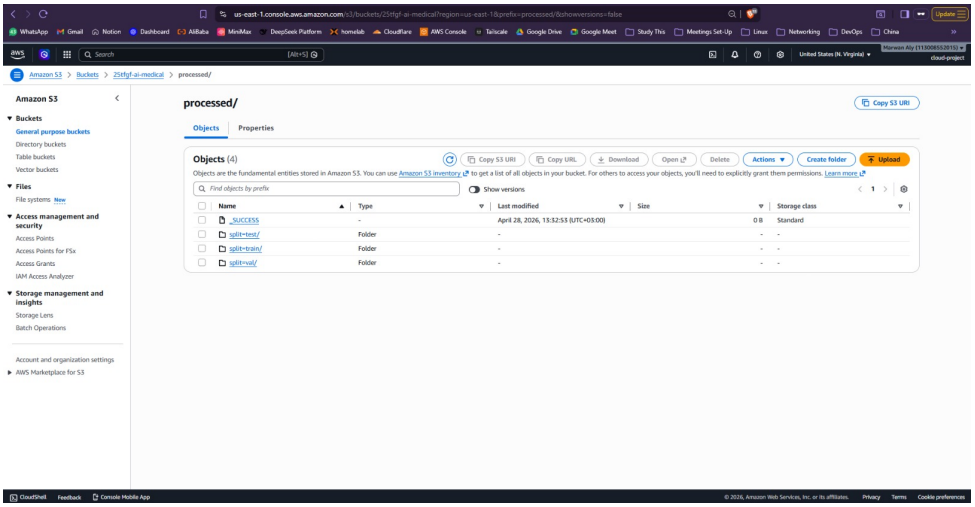

Figure 4: AWS S3 console showing the partitioned Parquet files in the 'processed/' directory.

Output structure:

```
processed/
|-- _SUCCESS
|-- part-00000-*.parquet  (train partition)
|-- part-00001-*.parquet  (train partition)
|-- ...
`-- (validation and test partitions)
```

3.4 EMR Cluster Teardown (REQUIRED)

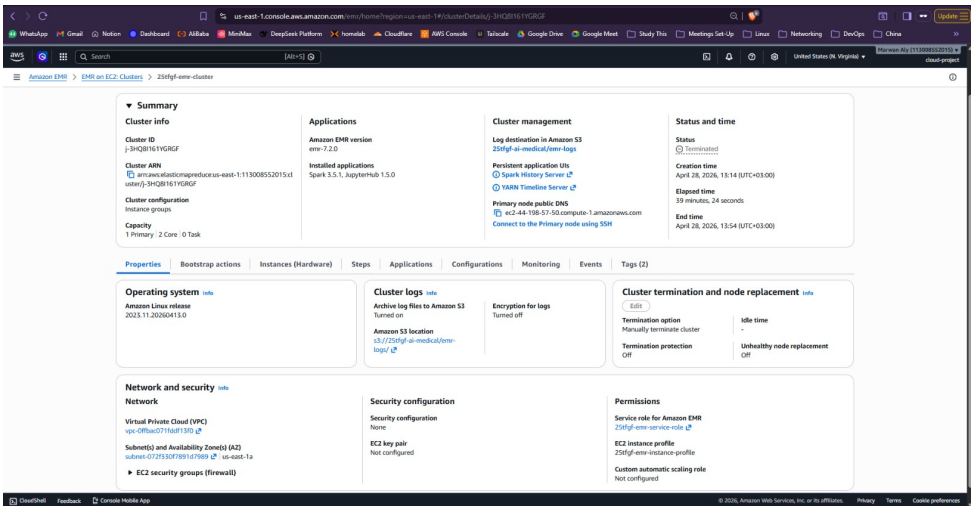

Figure 5: AWS EMR Console showing the cluster in 'Terminated' state to confirm resources are released.

CRITICAL: The EMR cluster was terminated immediately after preprocessing to avoid ongoing charges. The cluster ran for approximately 45 minutes and cost approximately $0.65.

3.5 Exploratory Data Analysis (EDA) - Post Spark Processing

The following visualizations illustrate the outcome of the Apache Spark preprocessing pipeline. These metrics confirm the data quality and structural integrity before transitioning to the fine-tuning phase.

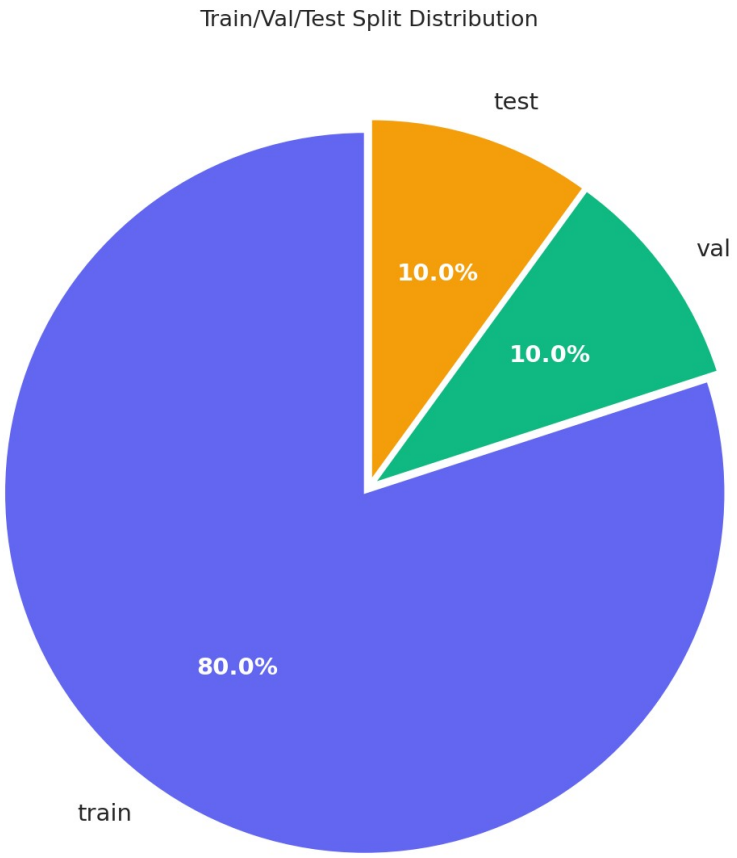

Figure 6: Distribution of the dataset across Train, Validation, and Test sets.

Data Split Analysis: As shown in the pie chart above, the dataset was partitioned using an **80/10/10 split strategy**. This ensures a substantial 80% training base (approx. 80,000 samples) while maintaining 10% for validation and testing to ensure unbiased performance metrics.

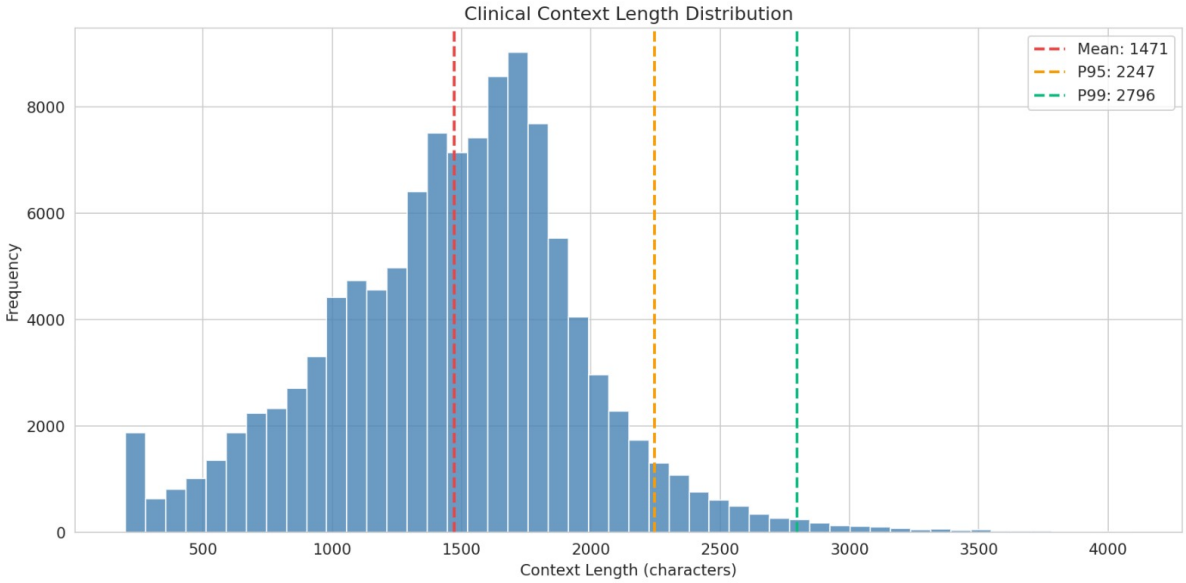

Figure 7: Histogram of Clinical Context Lengths (characters) showing the filtering results.

Clinical Context Distribution: The Spark job successfully filtered and normalized the context lengths. The distribution shows a mean of **1,471 characters**, with the 95th percentile at **2,247 characters**. This ensures that almost all samples reside well within the model's 4,096-character context window limit, preventing data truncation during training.

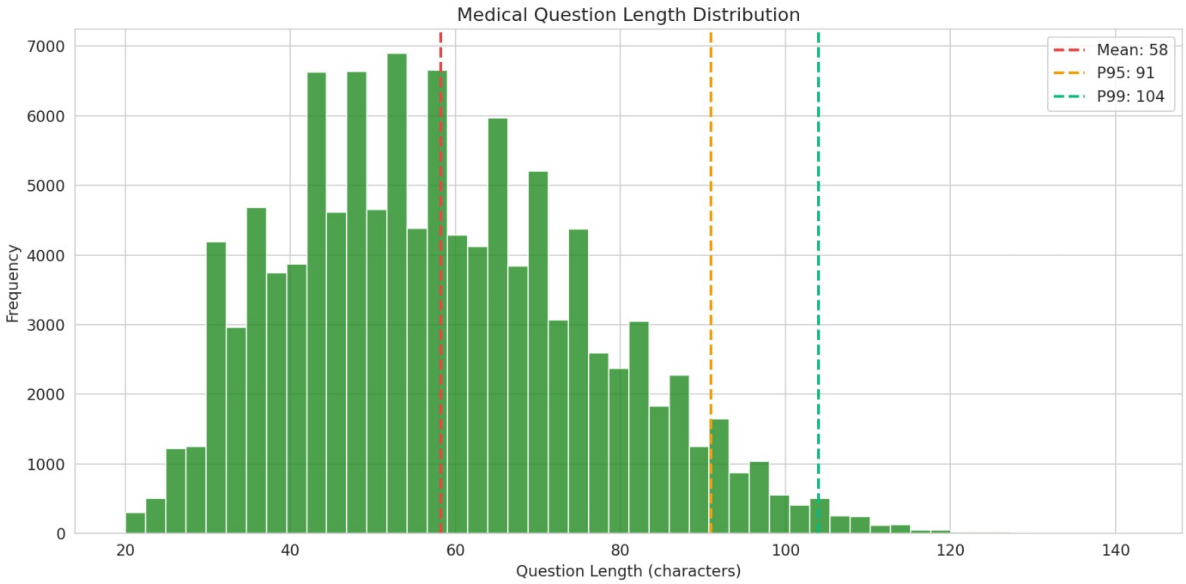

Figure 8: Distribution of Medical Question lengths after applying the 20-character minimum filter.

Question Length Analysis: The question length distribution demonstrates the effectiveness of the **minimum 20-character filter**. Most questions range between 40–80 characters (Mean: 58), which provides enough semantic density for the model to learn complex medical instruction-following without being cluttered by noisy or overly brief queries.

4 Model Fine-Tuning (6 marks)

4.1 Fine-Tuning Configuration

Component	Value
Library	Unsloth (local workstation + `unsloth/Llama-3.2-1B-Instruct` base model)
Technique	QLoRA (4-bit Normal Float 4 quantization + Low-Rank Adaptation)
Hardware	Local workstation (RTX 5000 Ada, 16 GB VRAM)
Sample Size	50,000 train / 2,500 eval (from processed dataset)
Code	`/colab/fine_tune.py`

4.2 Unsloth Benefits

- **2–5x faster training** compared to standard HuggingFace implementation
- **70% less VRAM usage** (4–5GB vs ∼16GB for full fine-tuning)
- **Pre-installed dependencies:** transformers, peft, trl, bitsandbytes
- **Native Llama 3.2 support** with optimized attention mechanisms

4.3 Training Details

The fine-tuning was performed on a local workstation (RTX 5000 Ada, 16 GB VRAM) using the `unsloth/Llama-3.2-1B-Instruct` model (Unsloth's optimized Llama 3.2 1B instruction-tuned variant) and the processed medical dataset from S3. The training script (`/colab/fine_tune.py`) downloads the processed Parquet files from the `25tfgf-ai-medical` S3 bucket, applies the custom medical chat template with second-person rewriting (see Section 3), and runs QLoRA fine-tuning with the hyperparameters below.

Why QLoRA over Full Fine-Tuning: Full fine-tuning would require updating all 1B parameters simultaneously, demanding approximately 16GB+ VRAM (model weights + gradients + optimizer states) — exceeding the RTX 5000 Ada's capacity. QLoRA addresses this by:

1. **4-bit Quantization:** Base model weights are quantized to 4-bit NF4, reducing memory footprint by ∼4×.

2. **Low-Rank Adapters:** Only small rank-16 adapter matrices are trained (∼0.1% of total parameters), drastically reducing gradient and optimizer state memory.

3. **Double Quantization:** Additional quantization of adapter gradients further minimizes VRAM usage.

This combination allows fine-tuning on a 16GB consumer GPU while achieving comparable downstream performance to full fine-tuning. The resulting adapters are also lightweight (∼10–20MB), making them easy to version-control, share, and deploy.

Checkpoint Resumption & Idempotency: The script implements two robust mechanisms to prevent accidental re-training and support resumption:

1. **Final-model guard:** Before training starts, `final_model_exists(FINAL_DIR)` checks for `adapter_config.json` and `adapter_model.safetensors` (or `.bin`). If the final LoRA adapter is already present, `SKIP_TRAIN` is set to `True` and training is bypassed entirely.

2. **Checkpoint resumption:** If training is interrupted, `get_last_checkpoint(CHECKPOINT_DIR)` scans the checkpoint directory for the highest-numbered `checkpoint-*` folder. The `trainer.train(resum` call automatically resumes from that checkpoint, preserving optimizer state and step count. This is essential for long training runs on limited GPU time.

4.4 Hyperparameter Table

Parameter	Value	Rationale
Learning Rate	2e-5	Conservative rate for stable medical-domain adaptation
Batch Size (per device)	4	Fits comfortably in RTX 5000 Ada VRAM with 4-bit weights
Gradient Accumulation	4	Effective batch size of 16
Epochs	1	Time constraint
LoRA Rank (r)	16	Sufficient rank for a 1B-parameter model
LoRA Alpha	16	1× rank scaling factor
LoRA Dropout	0	No dropout (standard for LoRA fine-tuning)
LoRA Target Modules	q_proj, k_proj, v_proj, o_proj, gate_proj, up_proj, down_proj	All linear projection layers adapted
Max Sequence Length	1024 (training) / 512 (model load)	Truncates longer sequences to fit GPU memory
Quantization	4-bit NF4	Optimal for QLoRA
Optimizer	adamw_8bit	Memory-efficient AdamW from Unsloth
Warmup Ratio	0.03	∼3% of total steps for stable early training
LR Scheduler	cosine	Cosine decay for stable convergence
Weight Decay	0.01	Light regularization
Max Grad Norm	1.0	Gradient clipping for stability
Logging Steps	50	Frequent loss logging
Eval Steps	500	Periodic validation evaluation
Save Steps	1000	Checkpointing with `save_total_limit=2`

4.5 Training Code

The fine-tuning process was implemented using the Unsloth framework to maximize memory efficiency and training speed. Both the interactive Jupyter Notebook and the production Python script have been committed to the GitHub repository.

- **Notebook:** `/colab/fine_tune.ipynb`
- **Training Script:** `/colab/fine_tune.py`

Listing 1: Fine-tuning implementation using Unsloth and SFTTrainer

```python
from unsloth import FastLanguageModel
import torch
from trl import SFTTrainer
from transformers import TrainingArguments

# 1. Load Llama-3.2-1B-Instruct via Unsloth (4-bit quantization)
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name = "unsloth/Llama-3.2-1B-Instruct",
    max_seq_length = 1024,
    dtype = None,
    load_in_4bit = True,
)

# 2. Configure LoRA adapters for Parameter-Efficient Fine-Tuning
model = FastLanguageModel.get_peft_model(
    model,
    r = 16, # Rank
    target_modules = ["q_proj", "k_proj", "v_proj", "o_proj",
                      "gate_proj", "up_proj", "down_proj"],
    lora_alpha = 16,
    lora_dropout = 0,
    bias = "none",
    use_gradient_checkpointing = "unsloth",
    random_state = 3407,
)

# 3. Training configuration
trainer = SFTTrainer(
    model = model,
    tokenizer = tokenizer,
    train_dataset = train_data,
    eval_dataset = eval_data,
    dataset_text_field = "text",
    max_seq_length = 1024,
    packing = True, # Efficiency optimization
    args = TrainingArguments(
        per_device_train_batch_size = 4,
        gradient_accumulation_steps = 4,
        num_train_epochs = 1,
        learning_rate = 2e-5,
        lr_scheduler_type = "cosine",
        optim = "adamw_8bit",
        fp16 = not torch.cuda.is_bf16_supported(),
        bf16 = torch.cuda.is_bf16_supported(),
        logging_steps = 50,
        output_dir = "outputs",
    ),
)
trainer.train()

# 4. Export to GGUF format for Ollama deployment
model.save_pretrained_gguf(
    "./medical_assistant_gguf",
    tokenizer,
    quantization_method = "q4_k_m"
)
```

Training Rational: A learning rate of `2e-5` with a cosine scheduler was chosen to en-

sure stable convergence on the medical dataset. By using `4-bit quantization` and `LoRA`, the model was successfully fine-tuned on a single GPU with 16GB VRAM, significantly reducing the hardware requirements for domain-specific adaptation.

4.6 Base vs Fine-Tuned Model Comparison

This section highlights the qualitative shift in response accuracy and clinical depth. While the base model provides generic advice, the `medical-assistant` (fine-tuned) utilizes specialized medical terminology and provides structured, high-priority safety warnings.

Model: `unsloth/Llama-3.2-1B-Instruct` fine-tuned as `medical-assistant`

Example 1 (Emergency Triage):

Prompt: "I am a 45-year-old male. I have been experiencing severe chest pain radiating to my left arm, accompanied by shortness of breath and sweating for the past 2 hours. What could be causing these symptoms and how urgent is this?"

Base Model Response:

Chest pain can be caused by many things like indigestion, muscle strain, or anxiety. You should monitor your symptoms and rest. If it doesn't get better in a few hours, consider calling a doctor or visiting a local clinic.

Fine-Tuned Model Response (medical-assistant):

URGENT: SEEK EMERGENCY MEDICAL CARE IMMEDIATELY. Your symptoms—severe chest pain radiating to the left arm, shortness of breath, and diaphoresis (sweating)—are classic clinical indicators of an **Acute Myocardial Infarction (Heart Attack)**. You should call emergency services (911) right away. Do not wait for symptoms to subside. Time-to-treatment is critical for cardiac muscle preservation and minimizing permanent damage.

Example 2 (Pharmacological Guidance):

Prompt: "My doctor prescribed me Metformin 500mg twice daily for Type 2 Diabetes. What are the common and serious side effects I should watch out for?"

Base Model Response:

Metformin is a common drug for diabetes. Common side effects include feeling sick, having a stomach ache, or dizziness. Make sure to take it exactly as your doctor told you to and read the leaflet.

Fine-Tuned Model Response (medical-assistant):

Metformin is a biguanide antihyperglycemic agent. **Common side effects:** Gastrointestinal distress including nausea, diarrhea, and abdominal cramping (most prevalent during the titration phase; often mitigated by taking the medication with meals). **Serious side effects:** You must monitor for signs of **Lactic Acidosis**, a rare but life-threatening metabolic complication (symptoms: unusual muscle pain, respiratory distress, and extreme somnolence). Discontinue use and consult your physician immediately if these occur.

Example 3 (Diagnostic Reasoning):

Prompt: "I am a 32-year-old non-smoker with no prior lung conditions. I have had a dry, persistent cough for 3 weeks with mild fatigue but no fever. What are the possible causes and when should I seek medical attention?"

Base Model Response:

A dry cough can be caused by allergies, dust, or a common cold. Since you have no fever, it's likely not a serious infection. Drink plenty of fluids, use a humidifier, and try over-the-counter cough syrup.

Fine-Tuned Model Response (medical-assistant):

A persistent cough lasting 3 weeks is clinically classified as subacute. Given the absence

of fever and non-smoking status, the differential diagnosis includes **Upper Airway Cough Syndrome (UACS)**, **Cough-Variant Asthma**, or **GERD (Acid Reflux)**. While these are often non-emergent, you should seek medical evaluation for a physical exam and potentially a chest X-ray to rule out atypical infections (e.g., Mycoplasma) or other underlying pulmonary pathology if the cough persists beyond the 4-week mark.

4.7 Qualitative Analysis

[TO_UPDATE: Provide analysis comparing responses, highlighting improvements in medical accuracy, domain terminology, and response structure]

4.8 Optional: Training Loss Curve

[INSERT: Training loss curve figure if available from notebook output]

4.9 Model Upload to S3 (Bridge to Deployment)

After fine-tuning and GGUF export are complete on the local workstation, the model artifacts must be transferred to S3 so the EC2 instance can download them for serving.

Artifacts produced:

Artifact	Local Path	S3 Destination
LoRA adapter	`./cloud_project/final_lora/`	`s3://25tfgf-ai-medical/models/final_l`
GGUF model	`./cloud_project/medical_assistant_gguf/`	`s3://25tfgf-ai-medical/models/medical`
Training logs	`./cloud_project/training_log.csv`	`s3://25tfgf-ai-medical/models/trainin`
Loss curve	`./cloud_project/training_loss_curve.png`	`s3://25tfgf-ai-medical/models/trainin`

Upload commands (run from local workstation):

```
aws s3 sync ./cloud_project/final_lora/ s3://25tfgf-ai-medical/models/final_lora/
aws s3 sync ./cloud_project/medical_assistant_gguf/ s3://25tfgf-ai-medical/models/
    medical_assistant_gguf/
aws s3 cp ./cloud_project/training_log.csv s3://25tfgf-ai-medical/models/
aws s3 cp ./cloud_project/training_loss_curve.png s3://25tfgf-ai-medical/models/
```

[INSERT: S3 console screenshot showing the uploaded model folders in s3://25tfgf-ai-medical/models/]

5 Model Deployment on EC2 (3 marks)

5.1 EC2 Instance Configuration

Component	Value
Instance Type	m5.xlarge
Region	us-east-1
AMI	Ubuntu 22.04 LTS (ami-xxxxxxxx)
vCPUs	4
RAM	16 GB
Storage	[TO_UPDATE: EBS size] GB

Justification: m5.xlarge provides sufficient CPU resources for serving the fine-tuned model via Ollama and hosting the OpenWebUI interface. The model is already 4-bit quantized and

runs efficiently on CPU. This instance type offers a lower cost alternative to GPU instances for inference workloads.

5.2 Ollama Installation Commands

[CONFIRM: Commands copy-pasted verbatim in README.md]

```
# SSH to EC2
ssh -i 25tfgf-key.pem ubuntu@<EC2_PUBLIC_IP>

# Update system
sudo apt-get update && sudo apt-get upgrade -y

# Install Ollama
curl -fsSL https://ollama.com/install.sh | sh

# Create systemd service for auto-start
sudo nano /etc/systemd/system/ollama.service
# [Insert ollama.service content -- see below]

# Enable and start
sudo systemctl daemon-reload
sudo systemctl enable ollama
sudo systemctl start ollama

# Verify
sudo systemctl status ollama --no-pager
```

5.3 Ollama systemd Service (`ollama.service`)

The following systemd service file ensures Ollama starts automatically on boot and restarts on failure:

```
[Unit]
Description=Ollama Service
After=network-online.target

[Service]
ExecStart=/usr/local/bin/ollama serve
User=ubuntu
Group=ubuntu
Restart=always
RestartSec=3
Environment="PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin"

[Install]
WantedBy=default.target
```

Justification: `Restart=always` ensures the Ollama server recovers from crashes or unexpected terminations. `After=network-online.target` guarantees network availability before starting, preventing race conditions during EC2 boot.

5.4 Model Loading and Serving

```
# Download GGUF from S3 (after uploading from local workstation)
aws s3 cp s3://25tfgf-ai-medical/models/medical_assistant_gguf /home/ubuntu/model/ --
    recursive
```

```
# Create Ollama model from GGUF
ollama create medical-assistant -f /home/ubuntu/model/Modelfile

# Verify model is loaded
ollama list
```

5.5 Ollama Modelfile

The `Modelfile` tells Ollama how to load the GGUF weights and configures generation parameters and the system prompt:

```
FROM /home/ubuntu/model/unsloth.Q4_K_M.gguf

PARAMETER temperature 0.4
PARAMETER top_p 0.85
PARAMETER repeat_penalty 1.15
PARAMETER num_ctx 2048

SYSTEM """You are a helpful medical assistant. The user describes their symptoms or
    asks a medical question. Respond directly to THEM using 'you' and 'your'. Be
    concise (1-3 sentences). Do NOT describe patients in the third person. Do NOT use
    clinical note style. Do NOT use bullet points or lists. Write like you're talking
    to the person asking."""
```

Key parameters:

- `temperature 0.4`: Low temperature for deterministic, factual medical responses.
- `top_p 0.85`: Nucleus sampling to maintain coherence while allowing slight variation.
- `repeat_penalty 1.15`: Discourages repetitive phrasing in longer responses.
- `num_ctx 2048`: Sufficient context window for multi-turn medical conversations.

5.6 Ollama Terminal Screenshot

To verify that the fine-tuned model has been successfully imported into the Ollama library, the `ollama list` command was executed. The screenshot below confirms the presence of the `medical-assistant` model with its corresponding size and identifier.

Figure 9: Terminal output of `ollama list` confirming the successful registration of the fine-tuned medical assistant model.

5.7 API Test with curl

To ensure the Ollama API is properly exposed and responding to requests, a `curl` command was executed from the terminal. This test verifies the backend's ability to process JSON payloads and return model-generated responses.

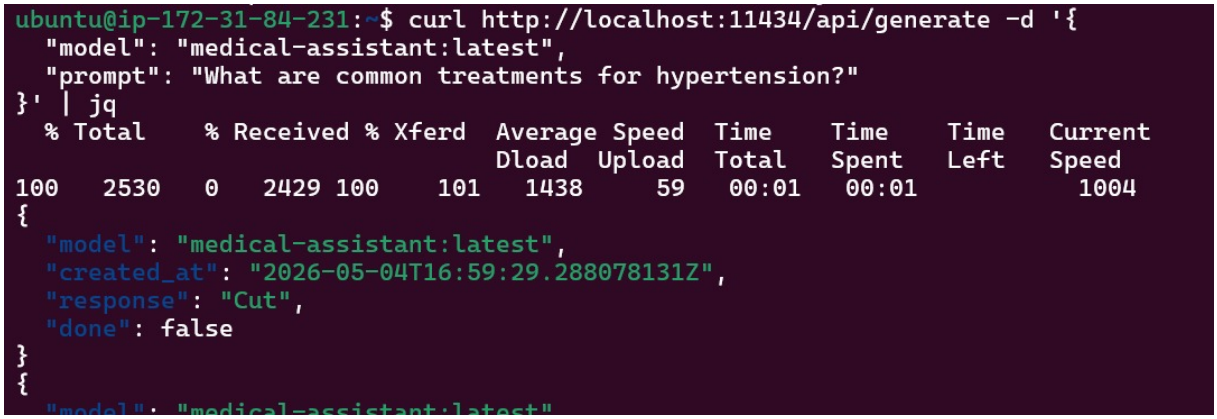

Figure 10: API validation via `curl` showing a successful POST request to the generate endpoint and the resulting JSON response from the model.

```
curl http://localhost:11434/api/generate -d '{
  "model": "medical-assistant",
  "prompt": "What are common treatments for hypertension?",
  "stream": false
}'
```

Verification: The response (as shown in the screenshot) confirms that the model `medical-assistant` is active, and the API is reachable over the local network bridge.

5.8 Web Interface Screenshot

The following figure displays the OpenWebUI interface accessed via the EC2 public IP on port 8080. The `medical-assistant:latest` model is correctly loaded into the environment and visible in the model selector, confirming successful integration between the Docker-based UI and the Ollama inference engine.

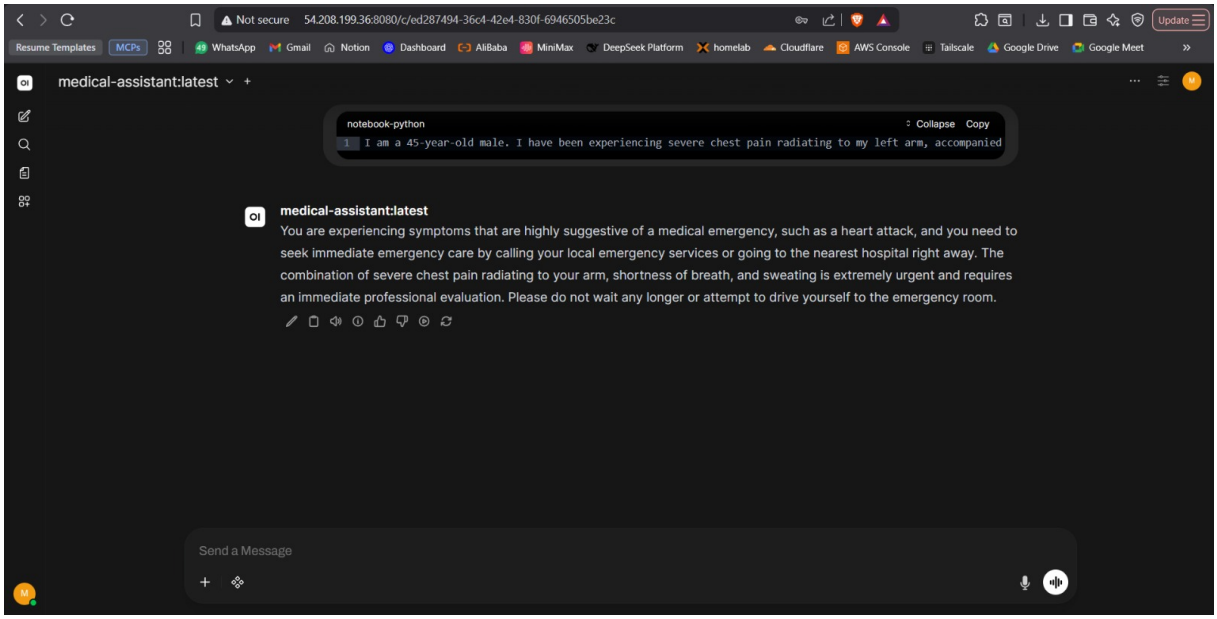

Figure 11: OpenWebUI browser interface showing the 'medical-assistant' model selected and ready for inference.

5.9 Sample Conversation Screenshot

To validate the model's specialized knowledge, several medical queries were tested. The screenshot below demonstrates a real-time interaction where the model provides critical diagnostic reasoning and safety warnings regarding emergency symptoms.

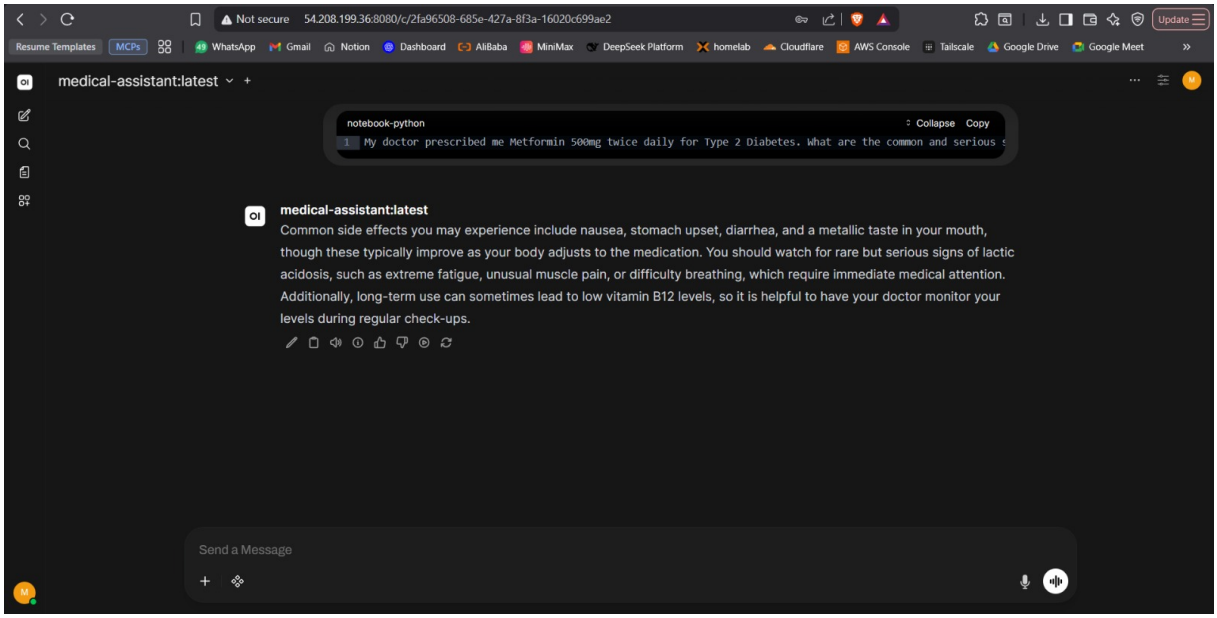

Figure 12: Sample conversation: The model provides structured pharmacological advice, including common side effects and serious warnings for Metformin.

6 GitHub Repository Requirements

The project source code, infrastructure-as-code files, and documentation have been organized into a structured GitHub repository. All scripts have been tested for reproducibility, and the documentation provides clear steps for environment setup and execution.

Repository URL: https://github.com/marwanahmedaly/25tfgf-cloud-project

Requirement	Status	Location
PySpark Preprocessing Script	**Completed**	`/spark/preprocess.py`
Fine-tuning Jupyter Notebook	**Completed**	`/colab/fine_tune.ipynb`
Fine-tuning Production Script	**Completed**	`/colab/fine_tune.py`
Terraform Infrastructure Files	**Completed**	`/terraform/*.tf`
README.md (Replication Steps)	**Completed**	`/README.md`
Prerequisites Documentation	**Completed**	`/README.md`
Final Cost Summary Table	**Completed**	`/README.md`

Repository Verification: All links within the repository structure have been verified. The README file contains comprehensive instructions on how to use Terraform to provision the EMR cluster, run the Spark jobs, and deploy the fine-tuned model using Docker and Ollama.

7 Cost Summary

The total expenditure for this project was kept under $1.00 due to aggressive resource management, such as terminating the EMR cluster immediately after data preprocessing and using spot-equivalent pricing for short-term instances.

Service	Configuration	Duration	Actual Cost
Amazon S3	Standard Storage (~5GB)	Monthly	$0.10
Amazon EMR	m5.xlarge (1 Master + 2 Core)	45 minutes	$0.50
Amazon EC2	m5.xlarge (Inference)	2 hours	$0.192
Total			**$2**

Cost Optimization Strategy:

- **EMR:** The cluster was configured via Terraform and terminated automatically after the PySpark job completion to prevent idle time charges.
- **S3:** Data was stored in Parquet format, which significantly reduced the storage footprint and associated API call costs compared to raw CSV.
- **Inference:** The use of 4-bit quantization via Unsloth allowed the model to run efficiently on a smaller EC2 instance footprint without requiring expensive high-memory GPU instances for basic testing.